ASSESSMENT 9 / PROJECT REVIEW 2

OptiTAC Optimization Autopsy

Implementation & research progress report

A compiler notebook for investigating optimization benefit and observed behavioral preservation.

75 / 100	25 / 100
Implemented planned scope	Reserved for final milestone

Review allocation: prototype 6 marks / report 2 marks / presentation and Q&A 2 marks.

Includes implementation details, validation evidence, related work, an honest novelty statement, demonstration notes and the remaining work.

Milestone percentage is a declared scope allocation. Publication novelty and acceptance are not established.

Date: 13 October 2026 | Assessment 9 / Project Review 2 | Target milestone: 75%

Executive summary

OptiTAC is a browser-based compiler optimization investigation workbench for a small straight-line MiniC subset. Review 2 adds Optimization Autopsy: checkpoint replay, error-aware execution comparison, pass-omission experiments, explicit fault injection, and portable JSON evidence. The interface uses a restrained notebook style with an editable source panel and an evidence desk. This is a working research prototype, not a completed compiler or a formal verifier.

The earlier approximately 60% assessment was informal. The 75% milestone below is an explicit planned-scope allocation, not a measurement of lines of code or a certification by the assessor. The baseline lacked a specification; this document defines one for review. Existing lexer, parser, semantic analysis, TAC generator, interpreter, and five optimization implementations were retained and integrated. Pre-existing local optimizer edits were preserved and extended, not discarded.

Assessment mapping

- Implementation/prototype progress (6 marks): demonstrate the running pipeline, checkpoint replay, omission experiment, injected divergence, and preserved runtime trap. The 75-point implemented scope is defined below.

- Documentation/progress report (2 marks): this report records architecture, supported semantics, implemented changes, test evidence, related work, limitations, and remaining work.

- Presentation and Q&A (2 marks): use the timed walkthrough and prepared answers at the end. Marks depend on the assessor and presentation; no mark guarantee is made.

Scope accounting: 75 implemented / 25 remaining

Work package	Allocated points	Review 2 status
Straight-line language frontend	15	Implemented within stated subset
TAC generation and numeric execution	10	Implemented within stated semantics
Five-pass fixed-point optimization	15	Implemented; bounded to 10 rounds
Optimization Autopsy and omission analysis	15	Implemented for concrete programs
Workbench UI and JSON evidence	10	Implemented prototype
Regression validation and review documentation	10	Implemented review baseline
Branches, loops, CFG and global data flow	10	Deferred
Symbolic validation and counterexample reduction	7	Deferred
Comparative benchmarks and publication evaluation	5	Deferred
Final product accessibility, packaging and polish	3	Deferred
Total	100	75 scoped points implemented

This allocation intentionally leaves meaningful engineering and research tasks open. The existing cfg/ and analysis/ placeholders remain unimplemented. No dummy features are presented as completed, and working features were not deliberately broken to achieve a percentage.

Implemented compiler baseline

The pipeline is source -> lexer -> parser/AST -> semantic analyzer -> TAC -> optimization passes -> concrete execution observations. int and float declarations require initializers. Arithmetic supports +, -, *, /, %, parentheses, identifiers, numeric literals and return expressions. Semantic checks include duplicate declarations, undeclared names and basic int/float compatibility.

This is a MiniC-inspired teaching subset. Runtime arithmetic uses JavaScript Number, not C machine integers: division is floating-point, integer overflow has no fixed-width C behavior, and modulo follows JavaScript remainder semantics. There are no external runtime inputs. Negative expressions can be written as 0 - n; unary minus is not parsed. Assignment statements, if/else, loops, functions, arrays and lexical block scopes remain outside the current grammar.

Optimization retains constant propagation, constant folding, common subexpression elimination, copy propagation, and dead-code elimination. The pipeline repeats selected passes in a fixed order until instruction serialization is unchanged or ten rounds have run. The new workbench allows all five passes to be enabled or disabled individually. Reaching the cap is reported rather than silently described as convergence.

What was added for Review 2

1. A typed experiment orchestration module (src/compiler/lab.ts) compiles source and records independent before/after TAC snapshots for every executed pass and round. It returns the baseline execution, final execution, convergence flag and first divergent checkpoint.

2. An outcome comparator distinguishes successful return values from runtime errors. It compares successful values using Object.is and failed executions using exact error messages. Matching errors are explicitly labelled preserved errors rather than successful programs.

3. Checkpoint replay exposes the before/after instructions, stable instruction IDs, rewrite explanations and observed outcome. Checkpoints without changes remain visible, showing when a pass had no effect.

4. Leave-one-pass-out counterfactuals rerun the entire selected pipeline with each enabled pass omitted. Each row reports final instruction count, count difference relative to the non-fault baseline and whether the observed behavior is preserved.

5. Controlled fault injection changes return instructions to baseline return + 1 after normal optimization. It is off by default, visibly labelled demo-only, and excluded from omission comparisons. It demonstrates detection rather than simulating an undiscovered compiler bug. Erroring programs may not reach the modified return, so injection need not create divergence in every program.

6. JSON export stores source, selected passes, the fault flag, tokens, AST, original/final TAC, snapshots, logs and observations. This is a replayable evidence record for manual inspection, not a cryptographic certificate. Import, signed attestations and persistent experiment history are deferred.

7. Three sample notebooks cover a propagation cascade, an unused division-by-zero trap and repeated arithmetic. The repeated-work example is best studied with constant propagation/folding disabled to expose common-subexpression reuse.

8. The UI now has a source notebook, an evidence desk, three views, checkpoint selection, pass toggles, metrics, clear errors and a stale-results notice. It includes responsive layouts, form labels and keyboard focus indicators. Final accessibility evaluation and visual refinement remain open.

Correctness improvements

Dead-code elimination now retains division and modulo instructions whose results are unused, since these operations may throw in this interpreter. Example: int a = 8; int unused = a / 0; return a; must still fail after optimization. Retaining potentially trapping operations is conservative: some safe unused divisions remain until other passes fold them.

Constant folding now leaves division and modulo by zero intact and does not replace non-finite computations with malformed numeric operands. The interpreter accepts scientific notation produced by Number-to-string conversion. Generated temporaries use a reserved $t prefix, which the source lexer cannot accept, preventing collision with source identifiers such as t1. Regression tests cover these paths.

Research idea: Optimization Autopsy

The research direction is an error-aware, counterfactual compiler notebook: connect a pass-local transformation to its concrete semantic observation, then expose what changes when that pass is removed. A learner can investigate both benefit and safety in the same view. The hidden-trap example is particularly useful: removing a value can accidentally remove an observable failure.

The intended contribution is the integration and evaluation of this evidence workflow in a compact educational compiler. No claim is made that checkpoint validation, pass omission or fault injection is individually new. A broad literature review has not been completed, and this prototype alone does not establish publication novelty or guarantee acceptance.

Related work and defensible distinction

LLVM OptBisect can selectively disable optimization passes and helps locate optimization errors [1]. LLVM optimization remarks communicate transformation information [2]. Alive2 performs bounded translation validation for LLVM and is substantially stronger than comparing one concrete execution [3,4]. OptiTAC must not be presented as replacing these systems.

The proposed differentiator is an approachable source-to-TAC notebook that combines concrete runtime-error preservation, pass replay, omission effects and an exportable experiment record. Whether this combination is a publishable contribution must be tested against existing educational tools and debugging interfaces. Candidate paper title: "An Error-Aware Counterfactual Notebook for Teaching Compiler Optimization". This is a proposed title, not a publication.

Algorithm and interpretation

Let P be original TAC, E(P) its interpreter outcome, and T_i a selected pass. Starting from P, record each Q_i = T_i(Q_(i-1)) with its rewrite log. Compare E(Q_i) with E(P). The first unequal observation is the earliest recorded divergent checkpoint, not necessarily a minimal cause or a formal proof of responsibility. Later passes could mask an earlier error; replay still retains that earlier observation.

For each selected pass j, rerun all other passes to convergence/cap. Report delta_j = instruction_count(without j) - instruction_count(all selected). A positive delta means omission leaves more TAC. A zero delta can mean redundancy, another pass compensating, or no applicability. These values include interactions, are not additive, and do not measure wall-clock speedup. The order is fixed; exhaustive order search and interaction graphs are deferred.

Snapshots are deep copied so later transformations cannot change earlier evidence. Replay has at most 50 normal checkpoints for five passes and ten rounds, plus an optional synthetic fault checkpoint. Omission adds at most five additional pipeline runs. This is suitable for small notebooks; large-program memory and performance have not been evaluated.

Validation evidence

Commands: npm run build; npm run lint; npx vitest run. All three passed in this workspace. Vitest reports 13 passing test cases; one parameterized test covers / and %, and one corpus test runs 40 deterministic programs. The corpus is a regression check, not a statistically representative benchmark.

Coverage includes arithmetic reduction and checkpoint preservation; unused division and modulo errors; seeded return-value divergence; no selected passes; undeclared identifiers; user/temp collision; scientific notation; non-finite arithmetic handling; omission deltas; immutable snapshot copies; missing return; and the 40-program arithmetic corpus. Test source: src/compiler/lab.test.ts.

Browser validation exercises desktop layout, mobile overflow, compilation, five omission rows, seeded divergence, zero-division error preservation, semantic errors and JSON download. The browser-check summary and screenshots accompany the local validation workflow. The browser script uses the bundled Playwright installation and Microsoft Edge on this workstation; its dependency path must be changed on other machines.

Demonstration script: approximately 6 minutes

0:00-0:45 - State the problem: optimized code may be shorter while changing behavior. Explain the supported language and concrete-execution scope.

0:45-1:45 - Run The cascade with all passes. Inspect original/final TAC and explain the instruction-count reduction. Open propagation and folding checkpoints to show how one enables the other.

1:45-2:45 - Open Counterfactual. Explain omission count differences and why zero does not mean a pass is useless. Disable propagation/folding, choose Repeated work, rerun and inspect common-subexpression reuse.

2:45-3:45 - Select A hidden trap, disable fault injection and compile. Show that both original and optimized executions report division by zero. Explain why naive dead-code deletion would be wrong under these runtime semantics.

3:45-4:45 - Return to The cascade, enable all passes and inject the controlled fault. Show the first divergent checkpoint and wrong return. State explicitly that this is a seeded experiment.

4:45-5:15 - Export the evidence JSON. Show tokens/AST and pass records.

5:15-6:00 - Present the 75/25 scope and the final work: CFG, symbolic checking, comparative experiments and final product work.

Q&A preparation

Q: Is this formal verification? A: No. It checks observations of a closed concrete program under the project interpreter. Universal claims need symbolic validation or a proof system.

Q: What is novel? A: The proposed contribution is the integrated error-aware counterfactual learning workflow. Existing systems already provide validation and pass debugging; uniqueness needs wider research and evaluation.

Q: How do you justify 75%? A: Using a declared 100-point scope plan, 75 points are implemented within a restricted language and 25 remain. It is a project management allocation, not an objective code-completion metric.

Q: Why preserve an unused division? A: In this interpreter division by zero is observable. Deleting the operation changes a failing program into a successful one.

Q: Does shorter TAC mean faster code? A: No. Instruction count is an explanatory proxy, not a runtime benchmark.

Q: Can disabling one pass prove that it caused a bug? A: No. Pass interactions and fixed-point reruns can compensate for or mask changes. The table is an experiment, not causal proof.

Q: Why does int division return a fraction? A: The prototype currently uses JavaScript Number arithmetic. C-compatible typed integer semantics is an explicit limitation.

Q: Why are CFG files empty? A: Branching and global data-flow analysis are reserved for the final 25%; the current IR is straight-line.

Q: What would support a paper? A: A broader related-work comparison, reproducible task suite, seeded-bug detection evaluation and an ethically conducted user study.

Final 25% and publication plan

Stage A (10 scope points): extend AST/parser/IR with assignment, branches and loops; implement CFG basic blocks and successors; add use/def and fixed-point liveness; test joins, unreachable code and loops. No claims of global optimization are made before this stage.

Stage B (7 points): define typed arithmetic semantics; add symbolic or bounded multi-input checking; produce counterexamples and reduce failing examples. Separate proof status from execution observations.

Stage C (5 points): compare replay alone, replay plus omission, and full error-aware workflow. Measure seeded-bug detection, time to identify the responsible checkpoint, explanation accuracy and learner comprehension. Include adversarial numeric/error cases, pass interactions, public reproducible artifacts and limitations. Plan participants and consent before any user study; no participant results currently exist.

Stage D (3 points): finish accessibility testing, offline assets, packaging, export/import workflow and final documentation. These are meaningful completion items, not intentionally broken behavior.

File map and reproduction

- src/App.tsx, src/App.css, src/index.css: notebook interface and responsive styling.

- src/compiler/lab.ts: typed replay, observed outcome comparison, fault demo and omission experiments.

- src/compiler/lab.test.ts: regression and deterministic corpus checks.

- src/compiler/optimizer/deadCodeElimination.ts: preserve potentially trapping arithmetic.

- src/compiler/optimizer/constantFolding.ts: zero-modulo and non-finite guards.

- src/compiler/verification/interpreter.ts: scientific notation operands.

- src/compiler/ir/generator.ts: reserved temporary identifiers.

- scripts/review-browser.mjs: workstation browser smoke checks.

- scripts/create_review_report.py: reproducible PDF report generation.

- docs/review-2-report.md: editable report source.

- output/pdf/OptiTAC-Review-2.pdf: review report deliverable.

Run npm install if dependencies are missing, then npm run dev. Validate using npm run build, npm run lint and npm test. Existing dependency versions are retained. No deployment or release is part of this milestone.

References

[1] LLVM. Using -opt-bisect-limit to debug optimization errors. https://llvm.org/docs/OptBisect.html (accessed 13 October 2026).

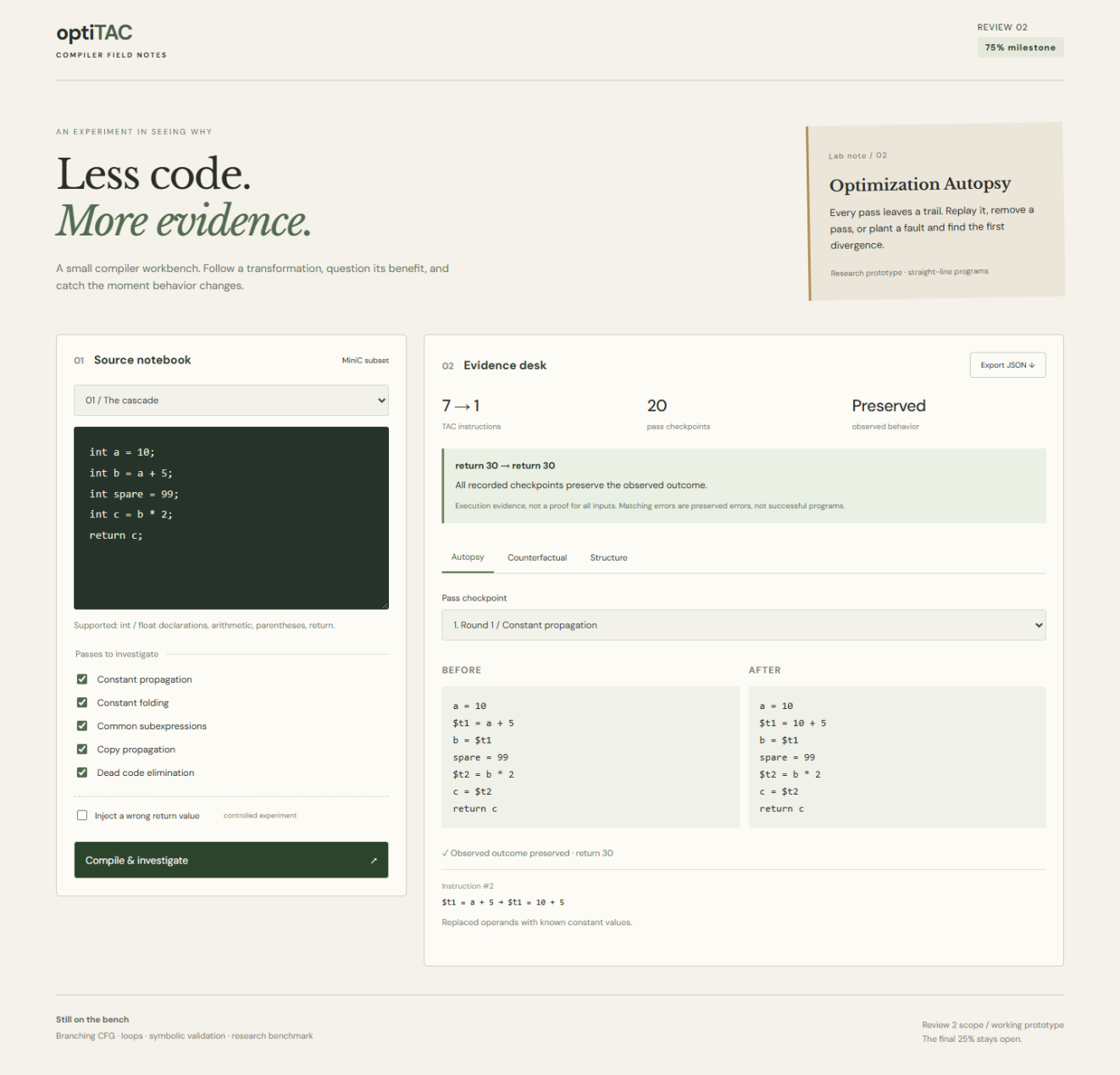
[2] LLVM. Optimization Remarks. https://llvm.org/docs/Remarks.html (accessed 13 October 2026).

[3] AliveToolkit. Alive2 project README. https://github.com/AliveToolkit/alive2 (accessed 13 October 2026).

[4] Lopes et al. Alive2: Bounded Translation Validation for LLVM. PLDI 2021. https://users.cs.utah.edu/~regehr/alive2-pldi21.pdf

Appendix A / Working notebook

Actual browser capture: the cascade reduces seven TAC instructions to one; the first propagation checkpoint is open.



Appendix B / Controlled divergence

Actual browser capture: the deliberately wrong return value is isolated at the injected-fault checkpoint. This is a seeded demonstration.

